

Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-214БВ-25

Студент: Сабирзянов А.Р.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 24.09.26

Москва, 2026

Постановка задачи

Вариант 18.

Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в одной из двух операционных систем. В результате работы программа (основной процесс) должен создать для решение задачи один или несколько дочерних процессов. Взаимодействие между процессами осуществляется через системные сигналы/события и/или каналы (pipe). Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы.

Общий метод и алгоритм решения

Для реализации межпроцессного взаимодействия были использованы два анонимных канала pipe. Родительский процесс считывает данные из STDIN_FILENO буфером и разбивает поток на отдельные строки, отправляя их поочередно в записывающие концы каналов pipe1[1] и pipe2[1].

Дочерние процессы создаются с помощью fork(). В каждом дочернем процессе перед выполнением функции execl() производится закрытие неиспользуемых концов каналов и перенаправление вывода: файловый дескриптор созданного/открытого файла дублируется на место STDOUT_FILENO с помощью dup2(). После этого происходит замена образа процесса на программу ./child.

Программа ./child в цикле считывает символы из канала (который стал для нее STDIN_FILENO), проверяет их на принадлежность к английским гласным и записывает разрешенные символы в STDOUT_FILENO (который указывает на файл).

Использованные системные вызовы:

pid_t fork(void); — создает новый дочерний процесс, дублируя вызывающий процесс.

int pipe(int pipefd[2]); — создает однонаправленный анонимный канал для межпроцессного обмена данными (pipefd[0] — для чтения, pipefd[1] — для записи).

int dup2(int oldfd, int newfd); — перенаправляет файловый дескриптор newfd на тот же файл, что и oldfd (используется для перенаправления stdout в файл).

int open(const char *pathname, int flags, mode_t mode); — открывает или создает файл для записи результатов работы дочерних процессов.

ssize_t read(int fd, void *buf, size_t count); — считывает данные из файлового дескриптора в буфер памяти.

ssize_t write(int fd, const void *buf, size_t count); — записывает данные из буфера памяти в файловый дескриптор.

int close(int fd); — закрывает файловый дескриптор, освобождая системный ресурс.

int execl(const char *path, const char *arg, ...); — заменяет текущий образ процесса новым исполняемым файлом.

pid_t waitpid(pid_t pid, int *wstatus, int options); — приостанавливает выполнение родительского процесса до завершения указанного дочернего процесса.

`void _exit(int status);` — немедленно завершает вызывающий процесс без выполнения обработчиков `atexit` и очистки буферов `stdio`.

Код программы

parent.c

```
#include <unistd.h>

#include <fcntl.h>

#include <sys/wait.h>

#include <stdlib.h>

#define BUFFER_SIZE 256

int read_line(char *buf, size_t max_len) {
    int bytes_read = read(STDIN_FILENO, buf, max_len - 1);

    if (bytes_read <= 0) {
        return 0;
    }

    buf[bytes_read] = '\0';

    if (bytes_read > 0 && buf[bytes_read - 1] == '\n') {
        buf[bytes_read - 1] = '\0';
    }

    return bytes_read;
}

int main() {
    char file1[BUFFER_SIZE];
    char file2[BUFFER_SIZE];

    if (read_line(file1, sizeof(file1)) <= 0) return 1;
    if (read_line(file2, sizeof(file2)) <= 0) return 1;
```

```
int pipe1[2];  
int pipe2[2];  
  
if (pipe(pipe1) == -1 || pipe(pipe2) == -1) {  
    return 1;  
}
```

```
pid_t pid1 = fork();  
if (pid1 == -1) return 1;
```

```
if (pid1 == 0) {  
    dup2(pipe1[0], STDIN_FILENO);  
  
    close(pipe1[0]);  
    close(pipe1[1]);  
    close(pipe2[0]);  
    close(pipe2[1]);  
  
    execl("./child", "./child", file1, NULL);  
    _exit(1);  
}
```

```
pid_t pid2 = fork();  
if (pid2 == -1) return 1;
```

```
if (pid2 == 0) {  
    dup2(pipe2[0], STDIN_FILENO);  
  
    close(pipe1[0]);
```

```

    close(pipe1[1]);

    close(pipe2[0]);

    close(pipe2[1]);


    execl("./child", "./child", file2, NULL);

    _exit(1);
}


close(pipe1[0]);

close(pipe2[0]);


char line_buf[BUFFER_SIZE];

int bytes;

int line_number = 1;


while ((bytes = read(STDIN_FILENO, line_buf, sizeof(line_buf))) > 0) {
    if (line_number % 2 != 0) {
        write(pipe1[1], line_buf, bytes);
    } else {
        write(pipe2[1], line_buf, bytes);
    }

    line_number++;
}


close(pipe1[1]);

close(pipe2[1]);


waitpid(pid1, NULL, 0);

waitpid(pid2, NULL, 0);

```

```
    return 0;
}
```

child.c

```
#include <unistd.h>
```

```
#include <fcntl.h>
```

```
#include <stdlib.h>
```

```
int is_vowel(char c) {
    return (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u' ||
           c == 'A' || c == 'E' || c == 'I' || c == 'O' || c == 'U');
}
```

```
int main(int argc, char *argv[]) {
```

```
    if (argc < 2) {
        _exit(1);
    }
```

```
    int fd = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0664);
    if (fd == -1) {
        _exit(1);
    }
```

```
    if (dup2(fd, STDOUT_FILENO) == -1) {
        close(fd);
        _exit(1);
    }
```

```
    close(fd);
```

```

char ch;

while (read(STDIN_FILENO, &ch, 1) > 0) {

    if (!is_vowel(ch)) {

        write(STDOUT_FILENO, &ch, 1);

    }

}

return 0;

}

```

Протокол работы программы

```
adelshck@DESKTOP-OJK8TKO:~/OS$ gcc -Wall parent.c -o parent
```

```
adelshck@DESKTOP-OJK8TKO:~/OS$ gcc -Wall child.c -o child
```

```
adelshck@DESKTOP-OJK8TKO:~/OS$ ./parent out1.txt out2.txt
```

Hello World

Operating Systems

POSIX system calls

Testing IPC Pipes

```
adelshck@DESKTOP-OJK8TKO:~/OS$ cat out1.txt
```

Hll Wrld

PSX systm clls

```
adelshck@DESKTOP-OJK8TKO:~/OS$ cat out2.txt
```

prtnng Systms

Tstng PC Pps

Вывод

В ходе выполнения лабораторной работы были успешно освоены базовые механизмы межпроцессного взаимодействия в операционных системах семейства Linux. Были на практике изучены системные вызовы pipe, fork, execl, dup2 и низкоуровневый ввод-вывод через read и write.